

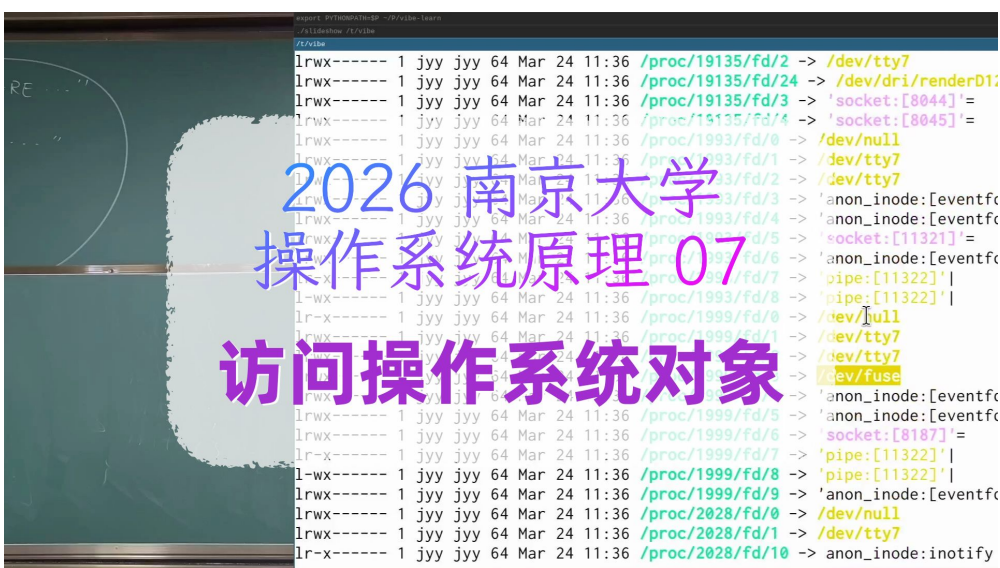
课程笔记

访问操作系统对象；文件描述符

南京大学 操作系统原理 2026 春季学期 第 07 讲

lecture-to-notes

2026 年 3 月 25 日



视频作者/频道: 绿导师原谅你了 (蒋炎岩 jyy)

发布日期：2026 年春季学期

视频时长: 1 小时 29 分钟

视频链接: <https://www.bilibili.com/video/BV1QFQmB7EQD>

目录

1	Everything is a File	2
1.1	操作系统对象 API 的设计	3
1.2	/proc 文件系统实战	4
1.3	本章小结	5
2	文件描述符	5
2.1	文件描述符 = 操作系统对象的“句柄”	5
2.2	“句柄”的历史	6
2.3	本章小结	7
3	管道 (Pipe)	7
3.1	管道的基本概念	7
3.2	管道的实现: fork + pipe + dup2	8
3.3	本章小结	9
4	总结与延伸	9
4.1	讲者的核心信息	9
4.2	结构化提炼	9
4.3	跨章节关联	10
4.4	开放问题	10
4.5	拓展阅读	10

1 Everything is a File

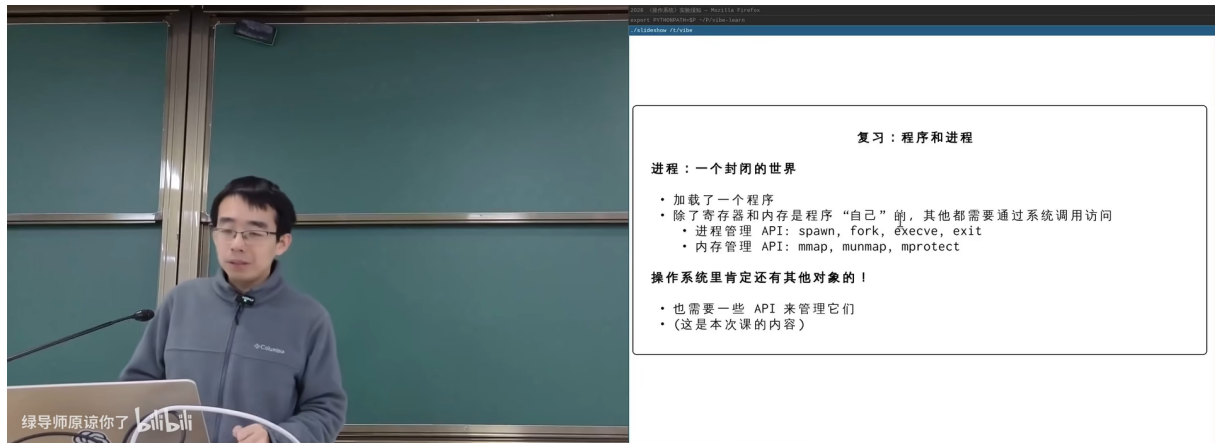


图 1: 复习：进程是一个封闭的世界；操作系统里还有其他对象需要 API 访问¹

jyy 从回顾开始：进程是一个封闭的世界——它只有自己的内存。要访问操作系统管理的其他资源（文件、网络、设备），需要新的 API。

UNIX 的核心哲学：Everything is a File

在 UNIX 中，几乎所有操作系统对象都被抽象为“文件”：

- 普通文件（磁盘上的数据）
- 目录（也是一种文件）
- 设备（`/dev/sda`, `/dev/null`, `/dev/urandom`）
- 进程信息（`/proc/[pid]/`）
- 网络连接（socket）
- 管道（pipe）

统一的接口：`open`, `read`, `write`, `close`, `lseek`——一套 API 操作所有对象。

¹ 视频画面时间区间：00:08:40–00:09:10。帧实际内容：slide “复习：程序和进程”——加载了一个程序，除了内存自己的状态和“自己”的东西，其他都需要通过系统调用访问。列出进程管理 API (`spawn`, `fork`, `execve`, `exit`) 和内存管理 API (`mmap`, `munmap`, `mprotect`)。提问：操作系统里还有其他对象吗？

1.1 操作系统对象 API 的设计

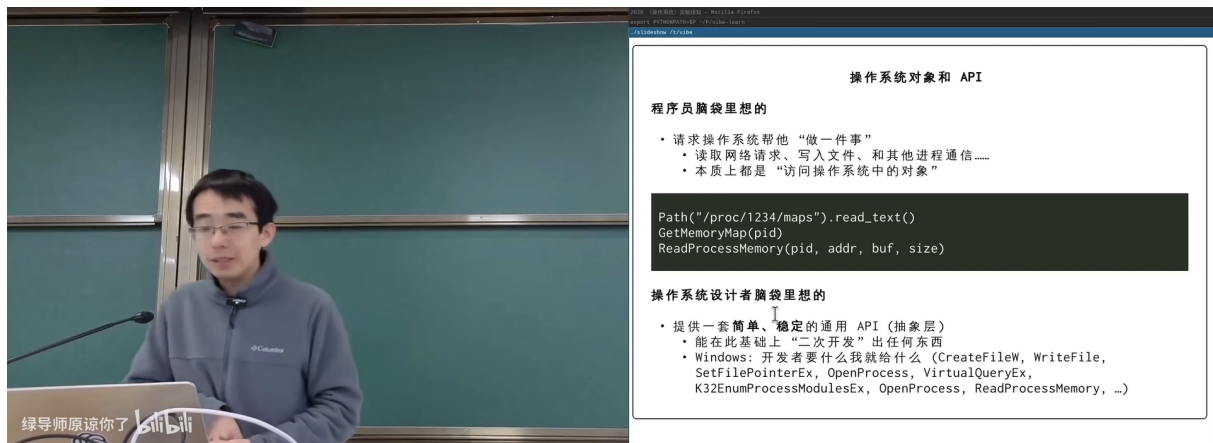


图 2: 操作系统对象 API: UNIX vs. Windows 的设计差异²

jyy 对比了两种操作系统对象 API 的设计哲学:

UNIX vs. Windows 的 API 哲学

UNIX: 统一只用“一把钥匙”——open/read/write/close 操作一切

- Path("/proc/1214/maps").read_text() — 读进程内存映射
- 文件、设备、进程信息——都用同一套接口

Windows: 每类对象有专用 API

- 文件: CreateFile, WriteFile, SetFilePointer
- 进程: OpenProcess, VirtualQueryEx, ReadProcessMemory

UNIX 的方式更简洁但有时受限; Windows 的方式更强大但 API 数量爆炸。

²视频画面时间区间: 00:16:10–00:16:40。帧实际内容: slide “操作系统对象 API”——程序员做什么的: 请求操作系统帮忙做“这一件事”(读取网络请求、写入文件), API 示例 Path("/proc/1214/maps").read_text() / GetMemoryMap(pid) / ReadProcessMemory(pid, addr, buf, size)。下半: 操作系统设计者做什么——“统一只用一把钥匙”, Windows: 开发者 + 数据库设计师 (CreateFile, WriteFile, ..., OpenProcess, VirtualQueryEx, ReadProcessMemory, ...)。

1.2 /proc 文件系统实战

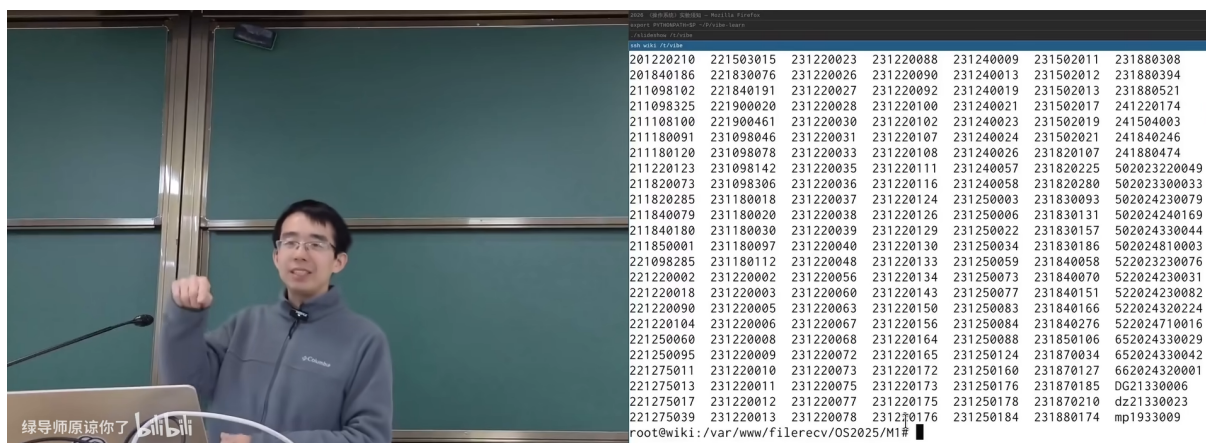


图 3: `/var/lib` 目录下的数字文件名——每个数字都是一个进程的信息³

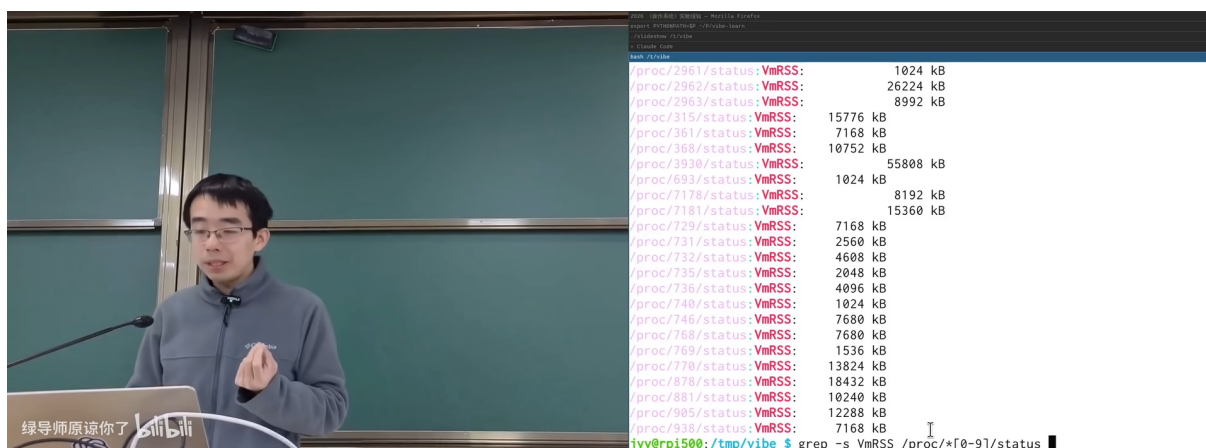


图 4: `grep -s VmRSS /proc/[0-9]*/status`: 一行命令查看所有进程的内存占用⁴

```
1 grep -s VmRSS /proc/[0-9]*/status
```

Listing 1: 一行命令查看系统中所有进程的内存占用

³ 视频画面时间区间: 00:26:10–00:26:40。帧实际内容: 终端显示大量数字命名的文件/目录 (20122021, 23158015, ...), 这是 `/proc` 或类似目录中以 PID 命名的进程信息。

⁴ 视频画面时间区间: 00:36:10–00:36:40。帧实际内容: 终端显示 `grep -s VmRSS /proc/[0-9]*/status` 的输出, 每行显示一个进程的 VmRSS (Resident Set Size) 值, 如 1024 kB, 26224 kB, 55808 kB 等。

/proc 的威力

因为“Everything is a File”，你可以用标准的文件操作工具（`grep`, `cat`, `awk`）来查询操作系统的内部状态：

- `grep VmRSS /proc/*/status` — 所有进程的内存占用
- `cat /proc/cpuinfo` — CPU 信息
- `cat /proc/meminfo` — 内存统计
- `ls /proc/[pid]/fd/` — 进程打开的文件描述符

不需要专用工具——`shell + grep` 就是最强大的系统监控工具。

1.3 本章小结

UNIX 的“Everything is a File”哲学将所有操作系统对象统一到 `open/read/write/close` 接口下。`/proc` 文件系统让你能用标准文件工具查询任何系统状态。

2 文件描述符

2.1 文件描述符 = 操作系统对象的“句柄”

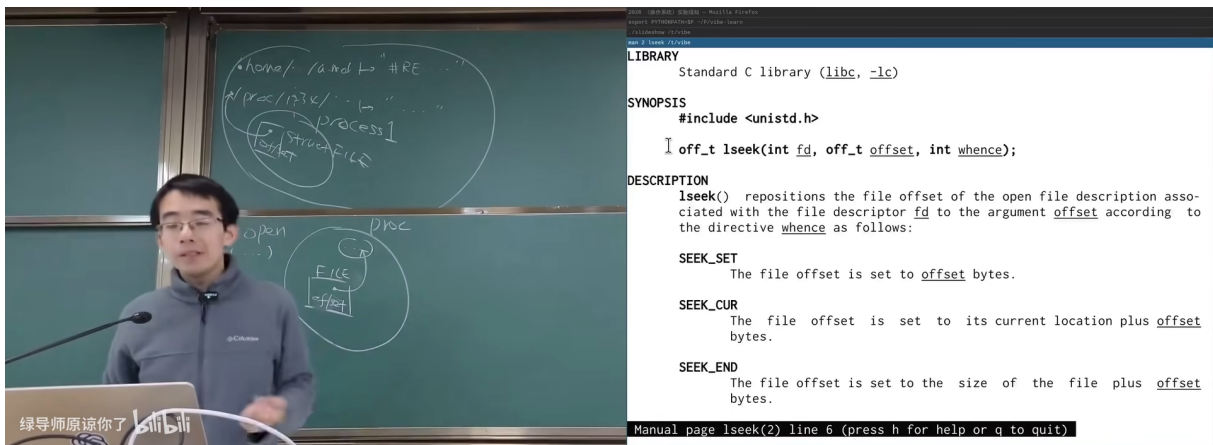


图 5: `man lseek(2)`: `SEEK_SET` / `SEEK_CUR` / `SEEK_END` 的语义⁵

⁵视频画面时间区间：00:47:11–00:47:41。帧实际内容：`man page` 显示 `lseek(2)` 函数签名 `off_t lseek(int fd, off_t offset, int whence)`，以及 `SEEK_SET`（设为 `offset`）、`SEEK_CUR`（当前位置 + `offset`）、`SEEK_END`（文件末尾 + `offset`）的描述。

文件描述符 (File Descriptor)

文件描述符是一个非负整数，代表进程打开的一个操作系统对象。

$fd = \text{open}(\text{path}, \text{flags}) \rightarrow \text{非负整数}$

标准约定：

- $fd = 0$ ：标准输入 (stdin)
- $fd = 1$ ：标准输出 (stdout)
- $fd = 2$ ：标准错误 (stderr)

核心操作：

- $\text{read}(fd, \text{buf}, n)$ — 从 fd 读取 n 字节
- $\text{write}(fd, \text{buf}, n)$ — 向 fd 写入 n 字节
- $\text{lseek}(fd, \text{offset}, \text{whence})$ — 移动文件偏移量
- $\text{close}(fd)$ — 关闭文件描述符

fd 是进程级的，不是系统级的

文件描述符是**每个进程独立**的。父进程的 $fd=3$ 和子进程的 $fd=3$ 可以指向不同的文件。但 fork 之后，子进程会**继承**父进程的所有 fd （指向相同的内核文件描述）。这就是为什么 fork 之后子进程能继续使用父进程的 stdin/stdout 。

2.2 “句柄”的历史

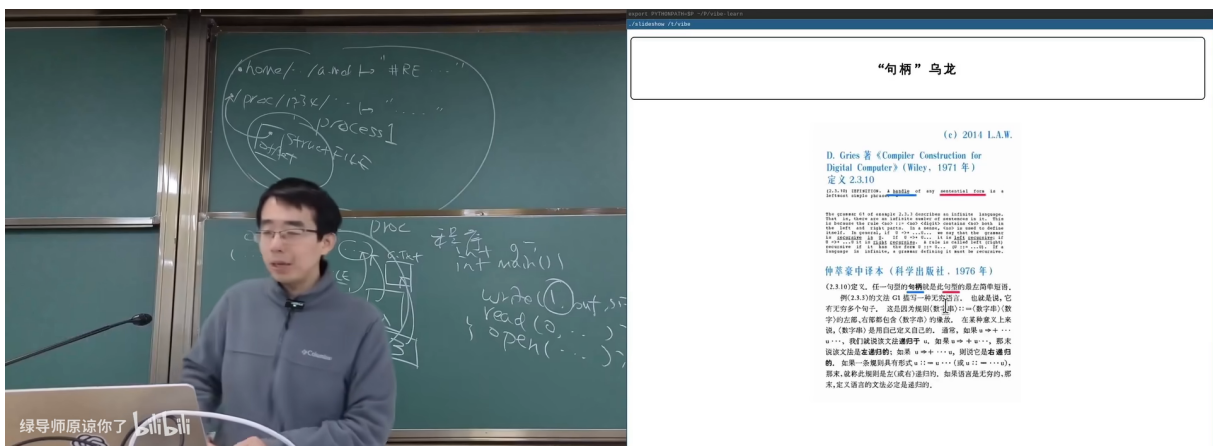


图 6: “句柄” 乌龙: D. Korn 的论文与文件描述符的起源⁶

⁶ 视频画面时间区间: 00:57:11–00:57:41。帧实际内容: slide “句柄乌龙”, 引用 D. Korn 《Complete Command to Digital Computers》(Micro, 2011), 讨论 “句柄” 这个概念的历史。下方引用关于 “数字索引” 设计的讨论。

为什么叫“句柄” (Handle)

“句柄”来自英文 handle——就像门把手。你不需要知道门背后是什么机制（锁芯、铰链），只需要抓住 handle 就能操作它。

文件描述符就是 UNIX 的 handle：你不需要知道 `fd=3` 背后是磁盘文件、网络 socket 还是管道，用 `read/write` 就能操作它。

2.3 本章小结

文件描述符是进程访问操作系统对象的统一“句柄”——一个非负整数。0/1/2 分别是 `stdin/stdout/stderr`。fork 后子进程继承父进程的所有 fd。lseek 控制文件偏移量。

3 管道 (Pipe)

3.1 管道的基本概念

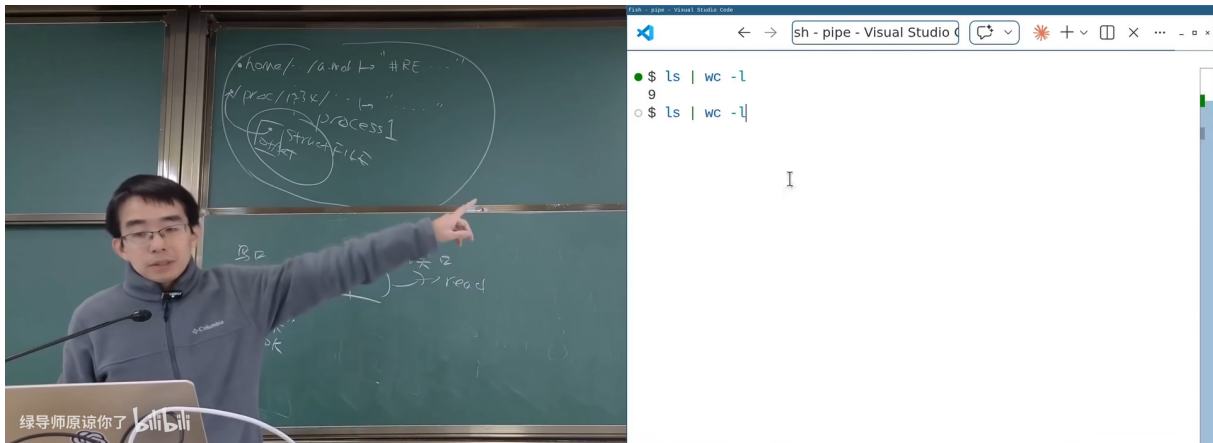


图 7: 管道演示: `ls | wc -l` 在 shell 中的效果⁷

管道

进程 A $\xrightarrow{\text{write}(\text{pipefd}[1])}$ Pipe Buffer $\xrightarrow{\text{read}(\text{pipefd}[0])}$ 进程 B

`pipe(int pipefd[2])` 创建一个管道:

- `pipefd[0]`: 读端 (read end)
- `pipefd[1]`: 写端 (write end)

Shell 中的 `|` 就是管道: `ls | wc -l` = 创建管道, fork 两个子进程, `ls` 的 `stdout` 连到管道写端, `wc` 的 `stdin` 连到管道读端。

⁷视频画面时间区间: 01:16:01–01:16:31。帧实际内容: VS Code 终端显示 `$ ls | wc -l` 输出 0, 以及 `$ ls | wc -l` 输出结果。jyy 在黑板上画了管道的示意图。

3.2 管道的实现: fork + pipe + dup2

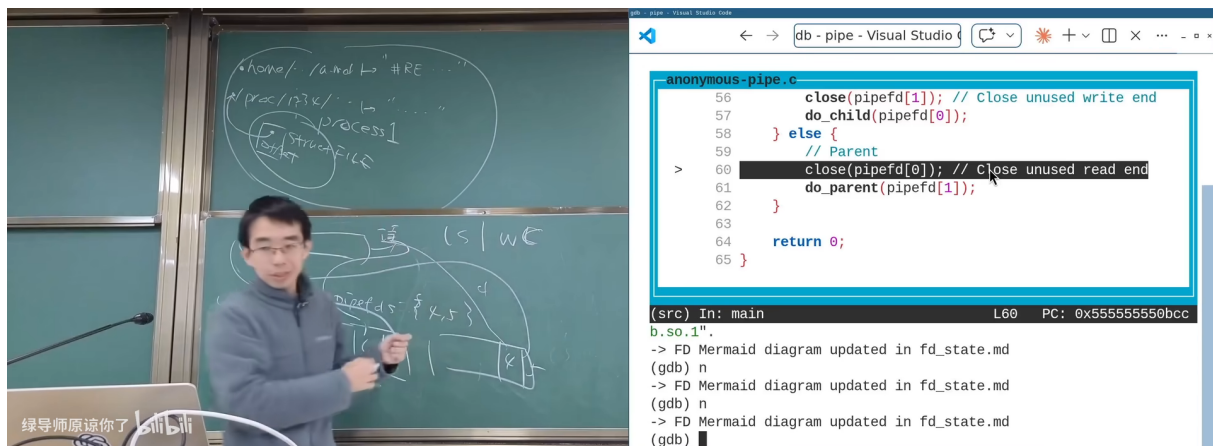


图 8: anonymous-pipe.c: 管道实现——close 未使用端 + do_child/do_parent⁸

```
1 int pipefd[2];
2 pipe(pipefd);
3
4 if (fork() == 0) {
5     // Child: read from pipe
6     close(pipefd[1]); // Close unused write end
7     do_child(pipefd[0]);
8 } else {
9     // Parent: write to pipe
10    close(pipefd[0]); // Close unused read end
11    do_parent(pipefd[1]);
12 }
```

Listing 2: 匿名字管道的典型使用模式

必须关闭未使用的管道端

fork 后父子进程都有 pipefd[0] 和 pipefd[1]。如果不关闭未使用的端:

- 子进程不关闭写端 → 管道永远不会收到 EOF → 子进程的 read 永远阻塞
- 父进程不关闭读端 → 如果父进程写满管道, write 会阻塞而非收到 SIGPIPE

规则: fork 后立即 close 自己不用的那一端。

⁸视频画面时间区间: 01:23:31–01:24:01。帧实际内容: VS Code 显示 anonymous-pipe.c 代码 (close/do_child/do_parent), GDB 调试器显示 “FD Mermaid diagram updated in fd_state.md”。

Shell 管道的完整实现

`ls | wc -l` 在 shell 内部的实现步骤:

1. `pipe(pipefd)` — 创建管道
2. `fork()` — 创建子进程 1 (`ls`)
3. 子进程 1: `dup2(pipefd[1], 1) + close(pipefd[0]) + execve("ls")`
4. `fork()` — 创建子进程 2 (`wc`)
5. 子进程 2: `dup2(pipefd[0], 0) + close(pipefd[1]) + execve("wc")`
6. 父进程: `close` 两端 + `waitpid` 等待两个子进程

`dup2(oldfd, newfd)` 的作用: 把 `oldfd` 复制到 `newfd` 的位置——让管道的读端“变成”`stdin`。

3.3 本章小结

管道是 UNIX 进程间通信的基础——一个内核中的字节流缓冲区, 通过 `pipe()` 创建, `fork` 后父子进程共享。Shell 的 `|` 运算符底层就是 `pipe + fork + dup2 + execve` 的组合。关闭未使用的管道端是避免死锁的关键。

4 总结与延伸

4.1 讲者的核心信息

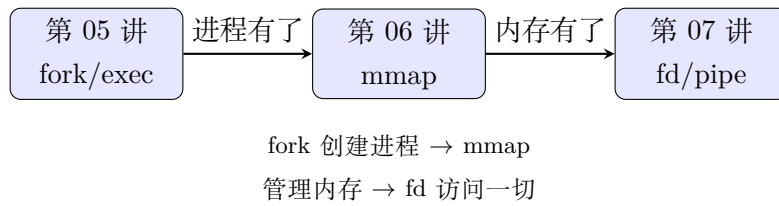
Everything is a File $\xrightarrow{\text{open}}$ fd (句柄) $\xrightarrow{\text{read/write}}$ 操作对象 $\xrightarrow{\text{pipe}}$ 进程间通信

4.2 结构化提炼

四个核心 Takeaway

1. **Everything is a File**——UNIX 把所有 OS 对象统一到文件接口下, 一套 `open/read/write/close` 操作一切。
2. **文件描述符 = 整数句柄**——进程通过 fd (非负整数) 引用 OS 对象。0/1/2 是 `stdin/stdout/stderr`。fd 是进程级的, `fork` 后继承。
3. **/proc 让系统透明**——标准文件工具 (`grep/cat/awk`) 就能查询任何系统状态。
4. **管道 = pipe + fork + dup2**——Shell 的 `|` 底层就是这三个系统调用的组合。必须关闭未使用端避免死锁。

4.3 跨章节关联



4.4 开放问题

1. dup2 和 dup 的区别是什么？何时用哪个？
2. 命名管道（FIFO）vs. 匿名管道有什么区别？
3. socket 也是 fd——网络编程如何统一到文件接口？

4.5 拓展阅读

- man 2 open, man 2 pipe, man 2 dup2
- OSTEP Chapter 39: Files and Directories
- jyy Wiki: <https://jyywiki.cn/>